CASE STUDY 5: REPEATED TRANSACTION DETECTION

1. Problem Statement

Repeated Transaction Detection is used to identify duplicate transactions in a list of financial transactions. Sometimes the same transaction may be recorded more than once. This program accepts transaction IDs from the user and identifies repeated transaction IDs using Python. It displays the duplicate transactions and helps detect repeated entries.

2. Objectives

- To develop a Python program to detect repeated transactions.
- To identify duplicate transaction IDs in a list.
- To display repeated transactions only once.
- To understand lists, loops, conditional statements, and sets in Python.
- To improve the accuracy of transaction records.

3. Assumptions

1. The user enters transaction IDs.
2. Each transaction ID is represented by a positive integer.
3. The user enters at least one transaction ID.
4. A transaction ID appearing more than once is considered repeated.
5. Each repeated transaction ID is displayed only once in the result.

4. Algorithm

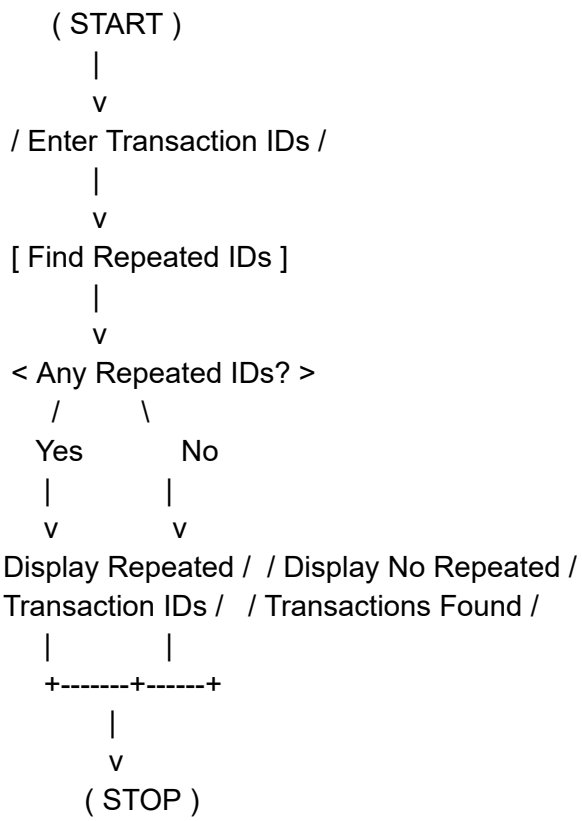
1. Start the program.
2. Ask the user to enter the number of transactions.
3. Accept the transaction IDs and store them in a list.
4. Create an empty list to store repeated transaction IDs.
5. Check each transaction ID in the list.
6. If an ID occurs more than once and has not already been added to the repeated list, add it.
7. If repeated IDs are found, display them.
8. Otherwise, display "No Repeated Transactions Found".
9. Stop the program.

5. Flowchart

Flowchart symbols:

- Oval – Start/Stop
- Parallelogram – Input/Output
- Rectangle – Process
- Diamond – Decision
- Arrow – Direction of flow

Simple flowchart:



Draw Start and Stop using oval shapes, inputs and outputs using parallelograms, processing using a rectangle, and the decision using a diamond.

6. Pseudocode

```
START

INPUT number of transactions
INPUT transaction IDs
repeated = empty list

FOR each transaction ID
   IF transaction ID occurs more than once
     AND transaction ID is not in repeated
      ADD transaction ID to repeated
   END IF
```

```
END FOR

IF repeated is not empty
   DISPLAY repeated transaction IDs
ELSE
   DISPLAY "No Repeated Transactions Found"
END IF

STOP
```

7. Python Source Code

```
n = int(input("Enter number of transactions: "))

transactions = []

for i in range(n):
   tid = int(input("Enter transaction ID: "))
   transactions.append(tid)

repeated = []

for tid in transactions:
   if transactions.count(tid) > 1 and tid not in repeated:
      repeated.append(tid)

if repeated:
   print("Repeated Transactions:", repeated)
else:
   print("No Repeated Transactions Found")
```

8. Test Cases and Expected Outputs

Test Case 1: Repeated Transactions Found

Input:

```
Enter number of transactions: 5
Enter transaction ID: 101
Enter transaction ID: 102
Enter transaction ID: 101
Enter transaction ID: 103
Enter transaction ID: 102
```

Expected Output:

```
Repeated Transactions: [101, 102]
```

Test Case 2: No Repeated Transactions

Input:

Enter number of transactions: 4
Enter transaction ID: 201
Enter transaction ID: 202
Enter transaction ID: 203
Enter transaction ID: 204

Expected Output:

No Repeated Transactions Found

Test Case 3: All Transactions Repeated

Input:

Enter number of transactions: 4
Enter transaction ID: 301
Enter transaction ID: 301
Enter transaction ID: 301
Enter transaction ID: 301

Expected Output:

Repeated Transactions: [301]

Note: Run each test case separately. Enter the transaction IDs exactly as shown.

9. Output Screenshots

```python
n = int(input("Enter number of transactions: "))

transactions = []

for i in range(n):
    tid = int(input("Enter transaction ID: "))
    transactions.append(tid)

repeated = []

for tid in transactions:
    if transactions.count(tid) > 1 and tid not in repeated:
        repeated.append(tid)

if repeated:
    print("Repeated Transactions:", repeated)
else:
    print("No Repeated Transactions Found")
```

```
= RESTART: C:/Users/sirit/OneDrive/Desktop/Python-Activity-Based-Learning/case5.
py
Enter number of transactions: 5
Enter transaction ID: 101
Enter transaction ID: 102
Enter transaction ID: 101
Enter transaction ID: 103
Enter transaction ID: 102
Repeated Transactions: [101, 102]
>>>
= RESTART: C:/Users/sirit/OneDrive/Desktop/Python-Activity-Based-Learning/case5.
py
Enter number of transactions: 4
Enter transaction ID: 201
Enter transaction ID: 202
Enter transaction ID: 203
Enter transaction ID: 204
No Repeated Transactions Found
>>>
= RESTART: C:/Users/sirit/OneDrive/Desktop/Python-Activity-Based-Learning/case5.
py
Enter number of transactions: 4
Enter transaction ID: 301
Enter transaction ID: 301
Enter transaction ID: 301
Enter transaction ID: 301
Repeated Transactions: [301]
```

10. Conclusion

Thus, the Repeated Transaction Detection program was successfully implemented using Python. The program identifies duplicate transaction IDs and displays each repeated ID only once. It also handles cases where no duplicate transactions are present. This case study demonstrates the use of lists, loops, conditional statements, and list methods in Python.